

Project Write Up

OWASP-ZAP Web Application Vulnerability Scanning

Sreeram Gurumurthy Ramesh

Why I Did This Project

Instead of just reading about web application security testing, I wanted to really do it. It's one thing to read about XSS, clickjacking, or missing security headers in a textbook; it's quite another to see those problems appear in a live scan against an active application. I therefore set up a small lab at home and scanned a website for vulnerabilities using OWASP ZAP. I chose OWASP Juice Shop as my target since it's an app designed to be vulnerable, making it ideal for learning without compromising any actual systems or breaking any laws. Since the security sector uses it widely for training, I was aware of the results I would obtain.

My objective was simple: I had to set everything up on my own, perform a proper scan, understand what ZAP was telling me, and document everything like a true security assessment. I wanted to be able to point to this project and say, "I didn't just watch a tutorial, I actually did this myself, start to finish."

Setting Up the Lab

I had to create the environment before I could scan anything. In order to run Juice Shop as a container rather than installing numerous dependencies directly on my computer, I installed Docker Desktop and Java 17 because OWASP ZAP is based on the Java runtime. Because the application runs in its own isolated container and I can remove and rebuild it whenever necessary, using Docker made the entire setup much cleaner.

I downloaded the official OWASP Juice Shop image and launched the container, mapping it to port 3000, after Docker Desktop was up and running. I could access the application in my browser at <http://localhost:3000> once the container was up, and I verified that the storefront loaded correctly with products, a login page, and a shopping cart, exactly like a typical e-commerce website would.

Before I started scanning, I made sure the program was functioning properly. I learned how much setup is required to create even a small testing environment, even if this aspect initially felt challenging. It also made me realize how much effort actual penetration testers devote to planning and scoping before doing a single scan.

Running the Scan

I launched OWASP ZAP and set it to <http://localhost:3000> after Juice Shop was operational. In order to capture more of the traffic and sites that an automatic crawl would ignore, I didn't only rely on the automated scanner; I also manually navigated the application while ZAP was operating in the background as a proxy. This involved clicking through various sections of the store, adding items to the cart, and logging in.

A typical spider wouldn't be able to capture everything because Juice Shop is a JavaScript-heavy application (much of the material loads dynamically rather than through regular page loads). I thus used ZAP's AJAX Spider, which use an actual browser to navigate the application and locate pages that a standard crawler would overlook. This mix of manual browsing, the classic spider, and the AJAX Spider offered me considerably better coverage of the app than any single method would have on its own.

I let ZAP run its active scan against everything it had found when the scan and spidering were finished. I then went through each of ZAP's alerts one at a time, examining the danger level, ZAP's level of confidence in each result, the evidence it collected, and the recommended solution for each problem. Instead of taking the risk classification at face value, I clicked into each alert to read the description and see why ZAP had warned it. In order to have a clean record of the evaluation, apart from the raw ZAP session file, I exported everything into a complete ZAP HTML report and took screenshots of the main conclusions.

What I Found

Juice Shop is designed for educational purposes, so it seems reasonable that the scan didn't find anything serious, but it did reveal a number of realistic, typical problems that frequently arise in real-world applications. Here are my findings and the reasons they are significant:

- **Missing Content Security Policy (CSP) header:** The browser's built-in protections against script injection attacks, such as XSS, are reduced in the absence of this header. Leaving CSP out eliminates a helpful safety net because it essentially tells the browser which content sources (scripts, styles, and pictures) it may trust.
- **Cross-Domain Misconfiguration:** Due to the app's excessively permissive cross-origin access, untrusted sites may be able to interact with it more than they should. This type of setting error can allow pages that shouldn't have access to data to read it.
- **Absence of an anti-clickjacking header:** Some answers failed to instruct the browser to prevent the page from loading within a hidden iframe on another website. Without this safeguard, a hacker might fool a user into clicking on a fake page that loads the true application below.
- **Session ID displayed in the URL:** Session identifiers appeared in the URL parameters, which is dangerous since URLs may be shared, cached, or tracked without anybody being aware that a session token is present. The session could be taken over by

someone else if that URL leaks.

- **Private IP disclosure:** Responses indicated private IP addresses, which exposed details about the internal network. This isn't a big concern on its own, but during reconnaissance, it's the kind of information that lets an attacker create a picture of the surroundings.
- **Unix timestamp disclosure:** Some responses exposed raw timestamps that may be used by an attacker to piece together details about the server, like the creation or last update date of certain data.
- **Missing X-Content-Type-Options:** nosniff header – Without this, browsers can try to "guess" content types, which opens the door to certain content-sniffing attacks where a file is interpreted differently than intended.
- A couple of informational-level notes, like data being stored in the browser's session storage, and confirmation that this is a modern, JavaScript-driven web app. These aren't vulnerabilities by themselves, but they're useful context for anyone reviewing the app later.

Most of these findings were rated Medium or Low, and a few were purely informational. None of them was critical on their own, but taken together, they show the kind of small misconfigurations that, in a real production app, can add up to a much bigger risk. This is actually a pattern I've read about in real breach reports too. It's rarely one huge flaw. It's usually a handful of small gaps that an attacker chains together.

How I'd Fix These Issues

Part of doing this project properly meant not just listing what was wrong, but thinking through how I would actually fix it if this were a real application I was responsible for:

- Add a properly scoped Content Security Policy header instead of leaving it unset, starting strict and loosening only where the app genuinely needs it.
- Lock down CORS configuration so only trusted, specific origins are allowed instead of a wildcard.
- Add the X-Frame-Options or CSP frame-ancestors directive so the app can't be embedded in a hidden iframe.
- Move session identifiers into cookies (with the Secure and HttpOnly flags set) instead of passing them through the URL.
- Review server responses and configuration to make sure internal IPs and raw timestamps aren't being exposed unnecessarily.

- Add the X-Content-Type-Options: nosniff header across all responses to stop browsers from guessing content types.

What I Took Away From This

This project helped me actually understand what these headers and settings are for, instead of just knowing their names. I learned:

- How to safely stand up a vulnerable app locally using Docker, so I had a legal, controlled target to test against.
- How OWASP ZAP's automated scanner and AJAX Spider work together, especially on apps that load content dynamically with JavaScript.
- How to actually read and interpret ZAP alerts — not just seeing "Medium risk" but understanding why it's risky and what the fix looks like.
- How missing security headers, even ones that seem small, can weaken a browser's built-in protections against attacks like XSS and clickjacking.